



*Computação em
nuvem*

Azure Service Bus

PAULO R T CÂNDIDO

Azure Service Bus



Motivação Filas – Comunicação Assíncrona

Quando um sistema é dividido em vários microsserviços, fazer com que um serviço chame o outro diretamente via HTTP (comunicações síncronas) o tempo todo cria uma dependência perigosa: se o Serviço B cair ou ficar lento, o Serviço A também cai ou trava.

É aí que entram as filas do Service Bus, atuando como um intermediário (Message Broker).

Cenários Comuns de Uso em Microserviços

1. Desacoplamento Temporal e Resiliência

Se o Microserviço de Pedidos precisa avisar o Microserviço de Faturamento que uma compra foi feita, ele não precisa esperar o Faturamento responder. O serviço de Pedidos simplesmente coloca a mensagem na fila e segue em frente. Mesmo que o Faturamento fique fora do ar por alguns minutos para uma manutenção, as mensagens ficam guardadas em segurança na fila e serão processadas assim que ele voltar.

2. Nivelamento de Carga (Load Leveling)

Imagine uma Black Friday: o microserviço que recebe as vendas sofre um pico gigantesco de requisições. Se ele mandasse tudo direto para o banco de dados ou para um serviço interno pesado, o sistema colapsaria. Com a fila, as mensagens acumulam ordenadamente. O serviço de processamento (os *consumers*) pode ir retirando e processando os dados no seu próprio ritmo, protegendo a infraestrutura de sobrecargas.

3. Garantias de Entrega e "At-Least-Once"

O Azure Service Bus trabalha nativamente com o modelo de **Peek-Lock**. Quando um microserviço pega uma mensagem para processar, ele "tranca" essa mensagem para que ninguém mais mexa nela.

- Se o processamento der certo, o serviço avisa a fila, e mensagem retirada da fila (`Complete`).
- Se o microserviço crashar no meio do caminho, a trava expira e a mensagem volta para a fila automaticamente para ser tentada de novo.

Principais Recursos para Microserviços

Recursos nativos que resolvem problemas complexos de código:

- **Dead-Lettering (Fila de Erros):** Se uma mensagem falhar repetidamente (por erro de código ou dados corrompidos), o Service Bus a move automaticamente para uma sub-fila de mensagens mortas (DLQ). Isso evita que uma mensagem ruim trave o microserviço em um loop infinito.
- **Sessões de Mensagens (FIFO Garantido):** Filas por padrão tentam manter a ordem, mas com o processamento paralelo em microserviços escalados, isso pode se perder. As *Sessions* garantem que mensagens com o mesmo ID de sessão (ex: o mesmo ID de Pedido) sejam processadas estritamente na ordem em que chegaram e pelo mesmo container/instância.
- **Deteção de Duplicidade:** O Service Bus consegue ignorar mensagens repetidas enviadas dentro de uma janela de tempo configurável.

Queue x Topic

Enquanto as **Queues (Filas)** são perfeitas para o modelo *ponto-a-ponto* (um produtor manda, apenas um microserviço específico consome), se você precisar que uma mesma mensagem seja enviada para múltiplos microserviços ao mesmo tempo (modelo *Publish/Subscribe (publicador/assinate)*, como Faturamento e Estoque reagindo ao mesmo pedido), o ideal é evoluir o desenho para usar os **Topics** do Service Bus. Todos os microserviços que assinarem o Tópico receberão a mensagem.

Terraform

Arquivo main.tf em:

github.com/prtcandido/comprnuvem/serviceBus